❶ Identifies a test class that focuses on R2DBC components

❷ Imports R2DBC configuration needed to enable auditing

❸ Activates automatic startup and cleanup of test containers

❹ Identifies a PostgreSQL container for testing

❺ Overwrites R2DBC and Flyway configuration to point to the test PostgreSQL instance

❻ Builds an R2DBC connection string, because Testcontainers doesn't provide one out of the box as it does for JDBC

❼ Initializes a StepVerifier object with the object returned by OrderRepository

❽ Asserts that the Order returned has the correct status

❾ Verifies that the reactive stream completed successfully

Since those slice tests are based on Testcontainers, ensure the Docker Engine is running in your local environment. Then run the tests:

```
$ ./gradlew test --tests OrderRepositoryR2dbcTests
```

In the next section, you'll write tests for the web slice.

8.5.3 Testing REST controllers with @WebFluxTest

The WebFlux slice can be tested similarly to how you tested the MVC layer in chapter 3 and using the same `WebTestClient` utility you used for the integration tests. It's an enhanced version of the standard `WebClient` object, containing extra features to simplify tests.

Create an `OrderControllerWebFluxTests` class, and annotate it with `@WebFluxTest(OrderController.class)` to collect the slice tests for `OrderController`. As you learned in chapter 3, you can use the `@MockBean` Spring annotation to mock the `OrderService` class and have Spring add it to the Spring context used in the test. That's what makes it injectable.